Seminar: Software Engineering in the Age of AI

University of Haifa — Department of Information Systems

Clinical Lab Analysis System

A controlled, self-correcting generated-code pipeline for answering predefined clinical laboratory questions over MIMIC-III

Final Report

- Hala Hillou
- Anas Khoury
- Dana Nmarny
- Alon Engel

Submitted to: Prof. Iris Reinhartz-Berger

August 2026

Table of Contents

1. Research Question, Goals and Importance

1.1 Research question

How can laboratory test results recorded during a hospital admission be used to automatically and reliably answer a limited, well-defined set of clinical laboratory analysis questions about hospitalized patients?

1.2 The engineering problem

Raw MIMIC-III laboratory data is large (close to a million lab-event rows once merged with admissions) and hard to query ad hoc. Clinicians reviewing a single admission and researchers exploring lab patterns across a cohort both need quick, correct answers to a small set of recurring questions, such as which results during a stay were abnormal, what the latest value of a test is, or how a value trended over time. At the same time, the seminar's broader theme — software engineering in the age of AI — raises a second, harder problem: large language models can now write the code needed to answer such questions on demand, but code generated by a model cannot be trusted blindly, especially over medical data, because it may reference the wrong columns, silently return an empty or malformed result, or execute an unsafe operation. The project therefore treats “generate code to answer a question” and “make sure that generated code is correct and safe before trusting its output” as one combined engineering problem, rather than two separate ones.

1.3 Why it matters

- Clinical relevance: fast, correct answers about abnormal results, trends, and admission-level summaries support bedside review by a doctor and reproducible, exportable analysis by a researcher, without requiring either user to write SQL or pandas code themselves.
- Software-engineering relevance: the project is a concrete case study in building an auditable “generated code” system — one that shows its work (the code it ran), checks its own output (validation), attempts to repair itself (correction), and only cautiously extends into open-ended LLM code generation behind a sandbox. This is directly relevant to the seminar's subject: how to engineer software responsibly when part of the pipeline is AI-generated.
- Safety in a sensitive domain: healthcare data raises the cost of a wrong or hallucinated answer. Restricting the system to a fixed, validated question registry by default — and only optionally allowing free-form LLM-authored code inside a hardened sandbox — is itself a design decision about how much autonomy to give generated code in a high-stakes domain.

1.4 Goals and objectives

1. Support a fixed, precisely specified set of clinical laboratory questions (12 in the final system) over a cleaned MIMIC-III merge of ADMISSIONS and LABEVENTS.
2. For every question, automatically generate executable Python code, run it, validate its output against explicit structural and clinical rules, and apply a bounded, rule-based self-correction step when execution or validation fails.
3. Present the result together with the generated code, the validation outcome, and the correction attempts, so a user can audit how an answer was produced, not just see the answer itself.
4. Offer an optional controlled natural-language interface that either maps user requests to one of the supported question templates or, in the experimental AI mode, invokes the guarded sandboxed code-generation pipeline.

5. As a bounded, off-by-default experiment, evaluate whether a guarded, sandboxed LLM “programmer agent” loop (inspired by the concepts presented in AgentCoder and Self-Edit, see section 7) can safely extend the system to free-form questions without weakening the guarantees of the trusted default path.
6. To support different analytical workflows, the final system provides separate Doctor and Researcher workspaces that share the same trusted backend pipeline while exposing questions most relevant to each user role.

1.5 Expected contribution

The expected contribution is a small but complete reference implementation of a controlled "generate → execute → validate → correct → present" pipeline for automated clinical laboratory analysis. The system combines a single-source-of-truth question registry that keeps the user interface, code generator, validator, and execution pipeline synchronized; a deterministic validation and rule-based correction layer that detects and repairs a meaningful class of execution errors without requiring additional model calls; and a clearly separated, sandboxed experimental extension that demonstrates how the same engineering principles can be applied to LLM-generated code while maintaining safety through guardrails such as AST allow-listing, restricted builtins, execution timeouts, row limits, and deterministic output validation.

2. Engineering Background and Proposed Method

2.1 Proposed approach

The system is a layered, registry-driven pipeline. A user selects one of the supported clinical questions (or types a natural-language question that is routed to one); the system generates a short, readable Python snippet that calls a known data-preparation function with the user's parameters; it executes that snippet; it validates the result against structural rules (is it a DataFrame? does it have the expected columns? is it non-empty?) plus a per-question clinical rule (for example, “all rows must be flagged abnormal”); and, only if execution or validation fails, it applies a small rule-based correction and retries, up to a bounded number of attempts. The final result, the generated code, the validation message, and the full correction history are all shown to the user.

```
question id  ->  generate code  ->  execute  ->  validate  ->  correct (if needed)  ->
present
```

This constrained pipeline is the trusted default. A second, explicitly optional and off-by-default mode (“Advanced: AI writes code”) provides an alternative execution path that bypasses the fixed registry and invokes an LLM “programmer agent” that writes a pandas snippet for a free-form question; that snippet never runs directly — it is checked against an AST allow-list and then executed inside a restricted sandbox, **following the same generate → execute → validate → retry pipeline before anything is shown to the user.**

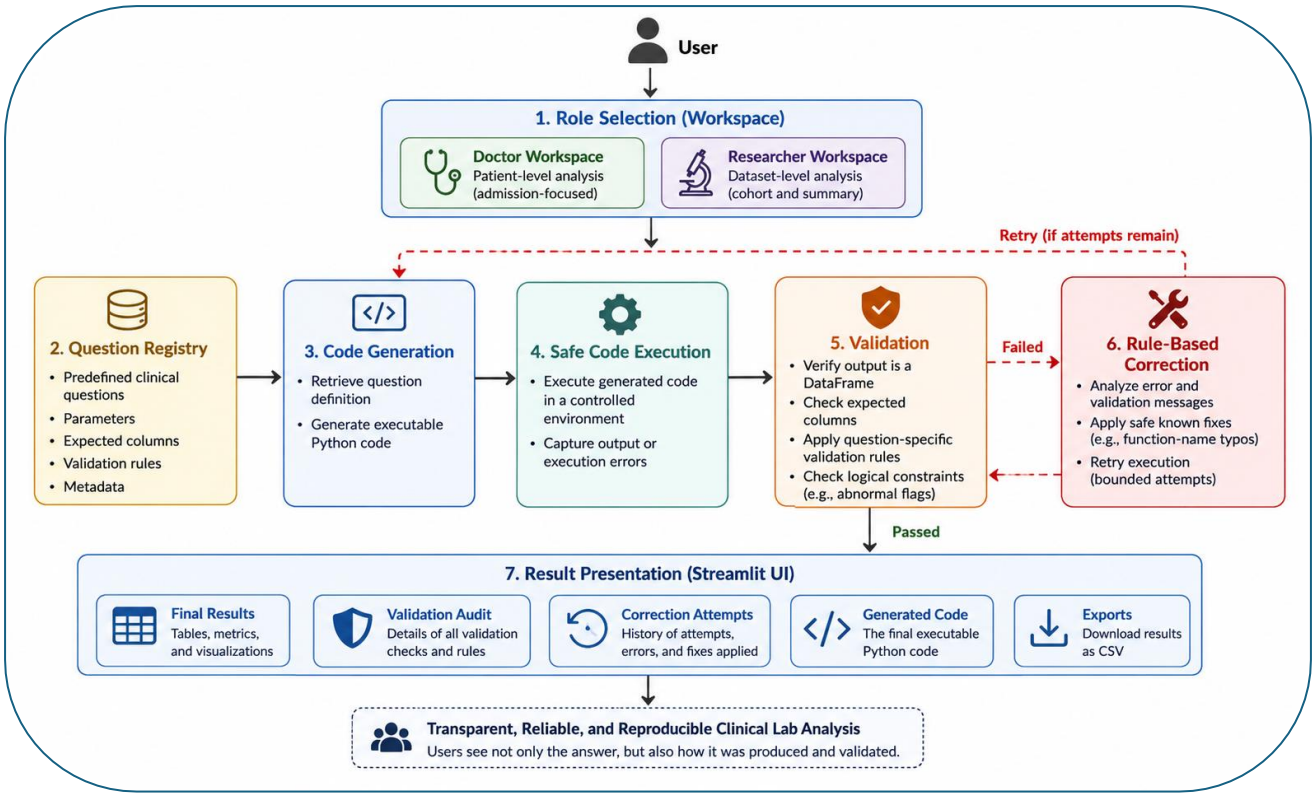
2.2 Architecture

The codebase is organized into single-responsibility modules, all driven by one configuration object: the question registry.

Module	Responsibility
data_prep.py	Loads the cleaned dataset once; one data function per supported question

Module	Responsibility
questions.py	QuestionSpec registry (QUESTION_REGISTRY) — the single source of truth: id, label, parameters, data function, expected columns, category, optional validator and chart spec
question_router.py	Controlled natural-language routing: maps free text to one supported question id + parameters (rule-based; never generates code)
llm_client.py / llm_router.py	Optional LLM-assisted routing fallback (Anthropic / OpenAI / Gemini, chosen by which API key is set); the model may only choose a template id + parameters, never write code
code_generation.py	Builds a runnable code string, e.g. result = get_lab_trend(107521, 51221), from a QuestionSpec + parameters
execution.py	Executes the generated code for the registry path and captures success / result / error / status
validation.py	Checks the result is a non-empty DataFrame with the expected columns, plus any per-question rule
correction.py	Bounded, rule-based retry loop: fixes a known set of function-name typos and re-runs
result_presentation.py	Console presentation of a pipeline run
main_pipeline.py	Wires generation → execution → validation → correction → presentation into run_pipeline(question_id, params)
app.py	Streamlit UI: role-based Doctor / Researcher workspace, question selection, dynamic parameter widgets, results, tabs
sandbox.py	AST allow-list + restricted-builtins execution environment for the optional LLM code-generation path
code_agent.py	The LLM “programmer agent”: builds a schema-aware prompt and asks the model for a pandas snippet
agent_pipeline.py	Guarded code-gen loop: programmer agent → sandbox → validator → refine, mirroring correction.py's shape
presentation_agent.py	Suggests a chart for a code-gen result (rule-based, optional LLM override)

The Question Registry acts as the central configuration layer of the system. All major components—including the user interface, code generation, validation, and execution pipeline—derive their behavior from the same registry definition, ensuring consistency across the entire application.



High-level architecture of the default, registry-driven pipeline (steps 1-7). "Safe Code Execution" here denotes the guarded try/except wrapper around exec() in execution.py for the trusted 12-question path; the further-hardened, AST-sandboxed executor (sandbox.py) that enforces an allow-list is used specifically by the optional Advanced "AI writes code" path described in section 2.3 and section 3.3.

2.3 Algorithms

Code generation

`generate_code(spec, params)` reads the target data function's own signature with Python's inspect module, so it needs no per-question hard-coded argument list. It builds a call such as `result = get_latest_lab_value(199884, 50902)` directly from the QuestionSpec and the supplied parameters.

Execution

For the registry path, `execute_generated_code` runs the snippet with Python's `exec()` against a namespace built only from the registry's known functions (never against arbitrary user code), inside a try/except that captures success, the result object, and any error message and status string.

Validation

`validate_result(spec, execution_result)` first verifies that execution completed successfully and returned a result. A missing result, an empty string, or an unsupported result type fails validation. A non-empty string is accepted as a valid textual result. For tabular results, the function verifies that the returned value is a pandas DataFrame containing all columns listed in the question's `expected_columns` and rejects an empty DataFrame. The Streamlit presentation layer handles this specific empty-result case separately and displays it as a neutral "no matching records" state. Finally, the validator applies the question-specific `extra_validate` rule, where one exists. Examples include verifying that abnormal-result questions return only FLAG = abnormal rows, that "latest value" and summary questions return exactly one row, that trend and date-range questions are sorted by CHARTTIME in ascending order, and that dataset-wide aggregate questions satisfy **Abnormal + Normal = Total Tests** for every admission.

Correction

run_with_correction executes and validates the code, and on failure calls correct_generated_code, which rewrites a small, explicit map of known function-name typos — matched on whole identifiers only, via regex word boundaries, so a wrong name that happens to be a prefix of a valid one is not corrupted — and retries, up to a bounded max_attempts (2 by default). Every attempt — its code, execution result, and validation result — is recorded and shown in the UI's Correction Attempts tab.

Relation to Self-Edit. This loop is a deliberate, rule-based implementation of Self-Edit's fault-aware editing pattern (Zhang et al., ACL 2023; section 7). The mapping is one-to-one: Self-Edit generates a program (our code generator), executes it against known checks (our execution and validation stages), turns the failure into a structured comment (our recorded execution error and validation message), and lets an editor rewrite the code before resubmitting (correct_generated_code, which rewrites the failed call and re-runs it, up to the bounded attempt limit). correction.py's own docstring names this lineage — "Correction module (simplified Self-Edit)". In a bounded 12-question registry the space of likely faults is small and known in advance, so a deterministic edit table replaces Self-Edit's trained neural editor while preserving the paper's generate → execute → feedback → edit → resubmit shape; the paper's finding that execution feedback alone repairs a meaningful share of errors is what justifies this cheap, deterministic design. The same pattern runs a second time — with a live LLM — in the Advanced mode's refine loop (agent_pipeline.py), where the model receives the sandbox's error feedback and edits its own snippet: a direct, unsimplified instance of Self-Edit's idea.

Because the trusted registry generator never produces a typo on its own, the UI includes a "Demo: inject a typo" checkbox (available for questions 1–5) that deliberately corrupts the generated function name. This lets an examiner trigger the correction mechanism on demand and watch the loop detect the failure, repair the name, and re-run — the same flow shown in Figure 6.

Controlled natural-language routing

The natural-language layer is a controlled interface in front of the main pipeline - not an open medical chatbot, and never a direct source of answers. Its only job is to translate a sentence written by the user into one approved question template and a validated set of parameters. The default router is a rule-based algorithm — deterministic keyword and regular-expression matching — rather than a trained model.

Where it is used. The natural-language input box in the Streamlit UI sends the user's sentence to the routing layer. question_router.py performs the default rule-based routing - text normalization, intent detection, parameter extraction, and mapping of lab names to ITEMID values - while llm_router.py acts as an optional fallback, used only when an API key is configured and the rule-based router cannot safely map the sentence.

Algorithm flow. The routing algorithm proceeds in five steps:

- Normalize the sentence and detect the intent - for example abnormal results, latest value, trend, summary statistics, counts, patient, admission, or date range.
- Extract entities such as HADM_ID, patient id, dates, and laboratory test names.
- Map lab names and aliases to known ITEMID values from D_LABITEMS.
- Validate the selected question id and parameters against QUESTION_REGISTRY and the QuestionSpec rules.

- Run the same trusted pipeline: generate Python code, execute it, validate the result, correct only if needed, and present the output.

How the LLM is controlled. In the normal routing path the LLM never writes code and never queries the dataset directly. It may only return structured JSON containing a supported question id and parameters; if the JSON is invalid, outside the 12-question registry, or missing required parameters, the system rejects it. AI is therefore used to understand and route the user's natural-language request - the medical result itself is always produced and checked by the deterministic pipeline.

Sandboxed LLM code generation (optional path)

check_code() parses a candidate snippet with Python's ast module and rejects it unless every node type is included in an explicit allow-list. Imports, loops, function and class definitions, private or dunder attribute access, dangerous builtins, and file-, network-, or evaluation-related methods such as to_csv, read_sql, eval, and query are rejected. Accepted code is executed by run_sandboxed() inside a worker thread in a restricted namespace containing only pd, the provided DataFrame df, and a small set of safe builtins. Execution is subject to a soft timeout, and DataFrame or Series outputs are capped to a configured maximum number of rows. The result then passes through the generic validate_freeform_result() gate. If execution or validation fails, the error feedback is returned to the LLM for another attempt, up to a bounded attempt limit.

2.4 Tools and technologies

- Python 3.10+ (developed and tested on Python 3.13), with pandas for data loading, preprocessing, and analysis, and Streamlit for the interactive web-based user interface, configured with a dark theme via .streamlit/config.toml.
- Automated testing using pytest and pytest-cov, with an 85% minimum code coverage threshold. The final implementation achieved approximately 94% code coverage.
- Optional LLM providers — Anthropic, OpenAI, or Google Gemini — selected according to the available API key in the .env file and accessed through a lightweight httpx-based client (llm_client.py) with automatic retries and a mock/none fallback, allowing the system to operate fully without external API calls.
- Dataset: The MIMIC-III clinical database, using the ADMISSIONS, LABEVENTS, and D_LABITEMS tables, cleaned and merged into a single working CSV file (see Section 3).

3. Implementation and Evaluation of the Solution

3.1 Inputs

The system is built on three MIMIC-III source tables: ADMISSIONS (one row per hospital admission, containing SUBJECT_ID, HADM_ID, ADMITTIME, DISCHTIME, and DIAGNOSIS), LABEVENTS (one row per laboratory measurement, containing SUBJECT_ID, HADM_ID, ITEMID, CHARTTIME, VALUENUM, VALUEUOM, and FLAG), and D_LABITEMS (a lookup table mapping ITEMID values to human-readable laboratory test names). During preprocessing, these tables are cleaned and merged into a single working dataset (cleaned_merged_dataset.csv), which is used by the application during runtime.

At runtime, the user selects one of the supported clinical questions and provides the required parameters (a subset of admission ID, laboratory test/ITEMID, patient ID, or date range), depending on the selected question. Alternatively, the user may enter a free-text natural-language question, which is routed to the appropriate predefined question through the controlled routing mechanism.

3.2 Data preprocessing

An initial schema-inspection pass (Google Colab, pandas) checked every field the supported questions rely on and quantified missing values in each table:

Table	Column	Missing values
ADMISSIONS	SUBJECT_ID, HADM_ID, ADMITTIME, DISCHTIME	0
ADMISSIONS	DIAGNOSIS	25 (non-critical for the MVP)
LABEVENTS	SUBJECT_ID / ITEMID / CHARTTIME	1 each
LABEVENTS	HADM_ID	371,649
LABEVENTS	VALUENUM	179,945
LABEVENTS	VALUEUOM	190,809
LABEVENTS	FLAG	994,847
D_LABITEMS	ITEMID, LABEL	0

Because the system joins LABEVENTS to ADMISSIONS on HADM_ID, rows with a missing HADM_ID cannot be attributed to an admission and are dropped. Rows with a missing VALUENUM are excluded only for the numeric analyses that require it (trend, latest value, first-vs-last, summary statistics); other questions (e.g. “all lab tests performed”) do not require it. FLAG has a large number of missing values because most lab results are normal and are simply left unflagged rather than explicitly marked “normal”; this is treated as expected, not as data corruption. The cleaning pipeline (1) keeps only the columns the project needs, (2) removes LABEVENTS rows with a null HADM_ID, (3) joins LABEVENTS to ADMISSIONS on HADM_ID, and (4) attaches a readable lab name from D_LABITEMS via ITEMID. The result, cleaned_merged_dataset.csv, contains 1,003,949 rows covering 2,839 admissions, 2,234 patients, and 376 distinct lab test types, with columns SUBJECT_ID_x, HADM_ID, ITEMID, CHARTTIME, VALUENUM, VALUEUOM, FLAG, SUBJECT_ID_y, ADMITTIME, DISCHTIME, DIAGNOSIS, LABEL. The file is kept out of version control because of its size and is fetched automatically by the app on first run (or placed manually next to app.py).

3.3 Main components

The trusted default path is the 12-question registry described in section 2, exposed through a role-based Streamlit UI: a Welcome screen lets the user enter a Doctor workspace (admission-level review — abnormal results, latest value, trend, first-vs-last comparison) or a Researcher workspace (dataset-level exploration — summary statistics, cohort counts, date-window filtering, CSV export), with questions relevant to both roles shown in both. Every question run produces a results area with summary metric cards, an optional trend chart, a colour-coded status (pass / no-records / fail), and four tabs: Final Result (with a CSV download button), Validation Audit, Correction Attempts, and Generated Code. A controlled natural-language box sits alongside the dropdown and is answered by the same registry through question_router.py, with an optional LLM-assisted fallback**.** The optional Advanced code-generation path provides an experimental execution path in which an LLM generates a pandas snippet for questions outside the fixed registry.

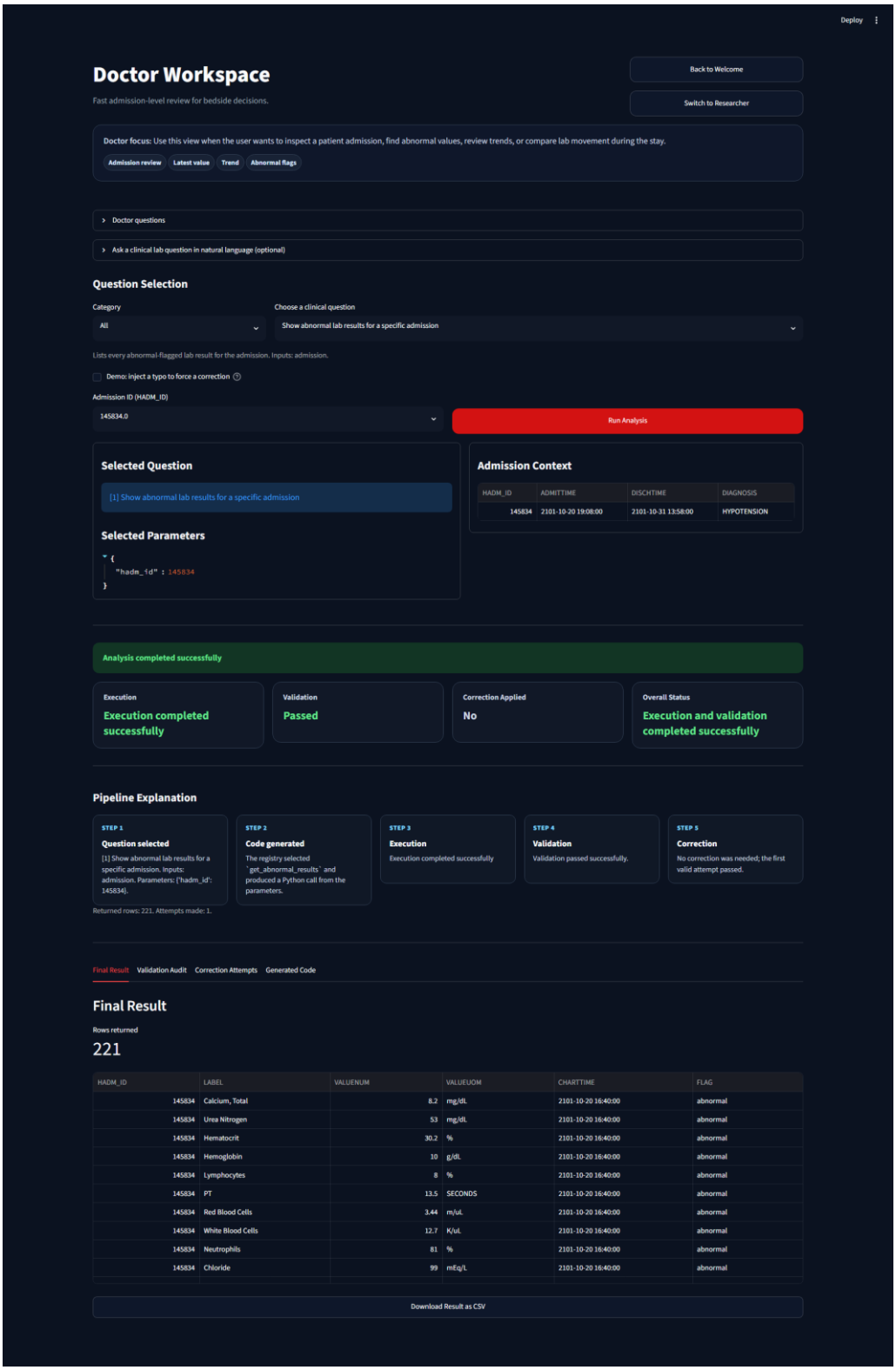


Figure 1. Question 1 ("Show abnormal lab results for a specific admission") in the Streamlit UI, with the Final Result, Validation Audit, Correction Attempts and Generated Code tabs.

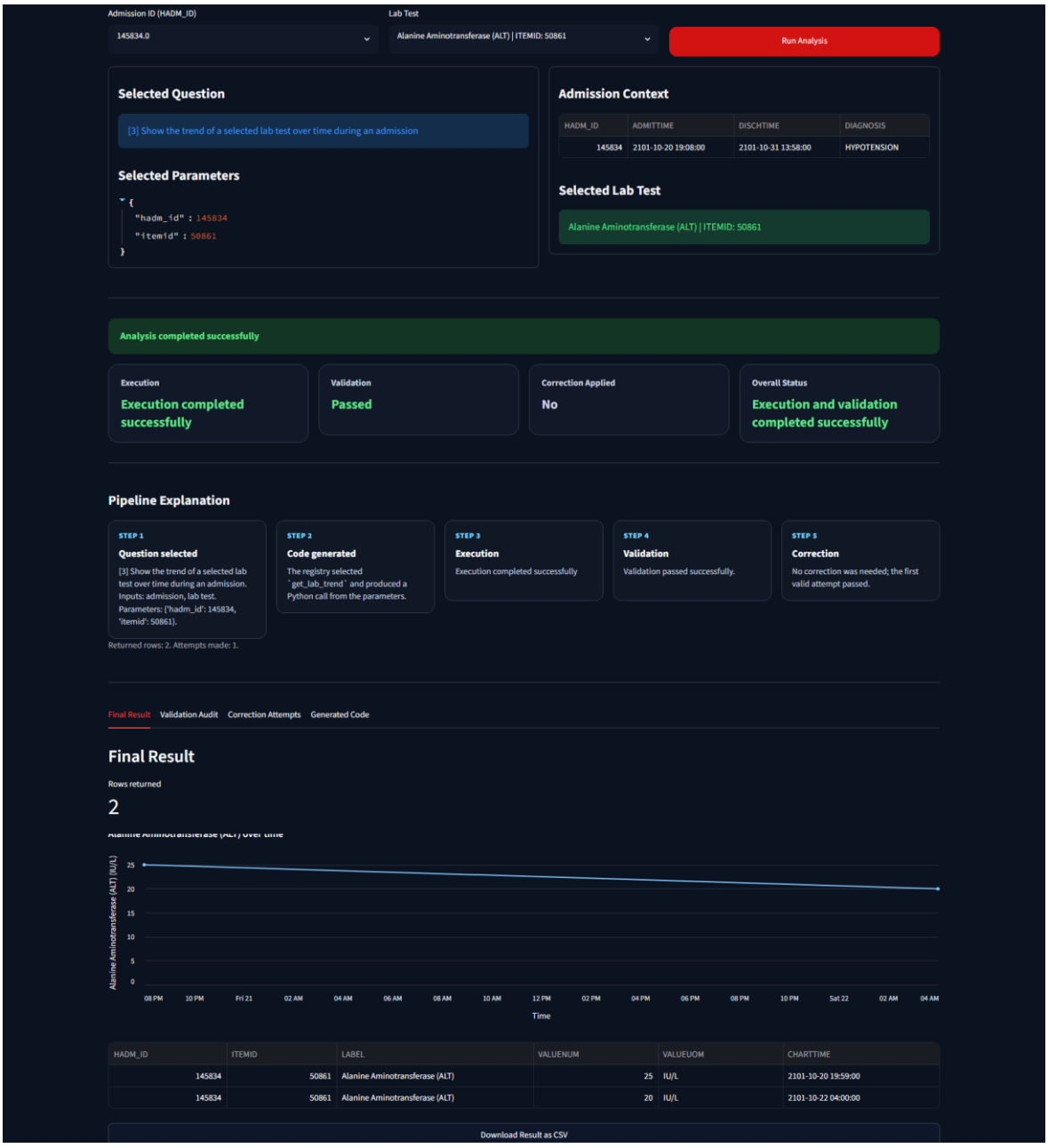


Figure 2. Question 3 (lab-value trend over an admission) rendered as a time-ordered table with an accompanying trend chart.

How the 12 supported questions were selected

The 12 supported questions were not chosen at random, and the LLM was not used as the authority for deciding the medical scope. The set was selected from the project requirements, the team's Part 1 data-analysis work, and the actual structure of the MIMIC-III laboratory dataset, using three selection criteria:

- Doctor workflow — narrow and specific. A clinician treating one patient needs precise answers about a single admission: which results are abnormal (Q1, and Q8 for one selected test), the latest value of a test (Q2), how it moved during the stay (Q3 trend, Q5 first-vs-last), and the admission's complete lab picture (Q4). These answer the bedside question "what is happening with this patient right now?"
- Researcher workflow — the bigger picture. A researcher looks across tests, patients, and time rather than at one bedside: summary statistics for a test (Q6), abnormal-versus-normal counts (Q7), the distinct abnormal tests ranked by frequency (Q9), date-window slices (Q10), a patient's full admission

history (Q11), and the dataset-wide aggregate (Q12). These answer “what patterns hold across the data?”

- Engineering constraint: every question must have clear inputs, expected output columns, and deterministic validation rules.

The selection was also grounded in background research. The team studied how Israeli health-fund portals present laboratory information to patients and clinicians — among them parts of Maccabi’s Health Guide (for example, its medical-conditions section at https://www.maccabi4u.co.il/healthguide/medicalconditions/) and Clalit’s Your Health portal (https://www.clalit.co.il/he/your_health/Pages/default.aspx) — to see which laboratory views real users are actually given: latest values, abnormal flags, and result trends over time. In parallel, the team ran iterative research sessions with an LLM (Claude), brainstorming, comparing, and refining candidate question sets across sessions; the resulting candidates were then filtered against the criteria above.

Because the project did not have a real doctor or researcher acting as a direct client, the target users were treated as two expected profiles. The Doctor profile needs fast admission-level review: what is abnormal, what is the latest value, and how a lab changed during the stay. The Researcher profile needs repeatable, exportable analysis: summary statistics, counts, date windows, patient-level context, and aggregate patterns across admissions. This is also why the UI is organized into Doctor and Researcher workspaces.

LLM assistance, where used, was limited to support tasks such as brainstorming, wording, and the optional natural-language routing; it did not decide which questions are clinically valid. Each question was accepted only if it satisfied four engineering criteria: it can be answered from the available dataset columns, it matches the project goal, it is meaningful for a doctor or researcher workflow, and its output can be validated with deterministic rules. The resulting set balances clinical usefulness, dataset feasibility, and software reliability - covering the main analysis patterns available in the data while remaining testable, reproducible, and safe to execute.

3.4 Results

Evaluation procedure. The system was evaluated in five complementary steps: (1) manual verification of every supported question type on multiple admissions and patients, checking the returned values against the source data; (2) an automated suite of 170 unit and integration tests covering every module; (3) end-to-end tests that drive the real Streamlit UI through all 12 questions in both role workspaces; (4) negative and edge-case testing of every failure path, including validation failures, no-record states, and sandbox rejection; and (5) internal baseline comparisons that ablate each guardrail (section 3.5). The subsections below report each step and its results.

Manual evaluation across multiple admissions and patients

To verify the correctness of the implemented questions, each supported question type was manually evaluated using multiple admissions and patients from the cleaned MIMIC-III dataset. The Table presents representative examples of these evaluations:

#	Example	Result
1	HADM 145834	Abnormal results returned (FLAG = abnormal)
2	HADM 199884, Chloride	Latest value 103 mEq/L

#	Example	Result
3	HADM 107521, Hematocrit	Decreasing trend, 57.9% → 44.4%
4	HADM 158675	All lab tests for the admission listed
5	HADM 177047, pO2	First 82, last 103, difference +21 mm Hg

Automated tests

170 automated tests pass with 94% line coverage (an 85% gate is enforced in pyproject.toml), covering every module plus an end-to-end test that drives the Streamlit UI (via AppTest) through all 12 questions, natural-language routing, and the experimental code-generation mode.

Module	Coverage
agent_pipeline.py / code_generation.py / correction.py / execution.py / llm_router.py / result_presentation.py	100%
sandbox.py	99%
data_prep.py	96%
main_pipeline.py / validation.py	95%
app.py / code_agent.py	94%
presentation_agent.py	92%
questions.py / question_router.py	91%
llm_client.py	90%
Total	94%

#	Question	Expected result	Status
1	Abnormal lab results	Table, all FLAG = abnormal	Pass
2	Latest lab value	Exactly 1 row	Pass
3	Lab trend over time	Time-sorted table + line chart	Pass
4	All lab tests	Table of all tests	Pass
5	First vs last value	1-row comparison incl. difference	Pass
6	Summary statistics	1 row: count / min / max / mean	Pass
7	Abnormal vs normal counts	1 row: total / abnormal / normal	Pass
8	Abnormal results for a lab	Abnormal rows, or graceful “no records”	Pass
9	Distinct abnormal tests	Table of tests with abnormal counts	Pass
10	Values in a date range	Time-sorted table + line chart	Pass
11	All admissions for a patient	Table of the patient's admissions	Pass
12	Abnormal vs normal, all admissions	One row per admission, most abnormal first	Pass

All twelve predefined clinical laboratory question types successfully passed both manual verification and automated validation, demonstrating that the registry-driven pipeline operates correctly across all supported analysis scenarios.

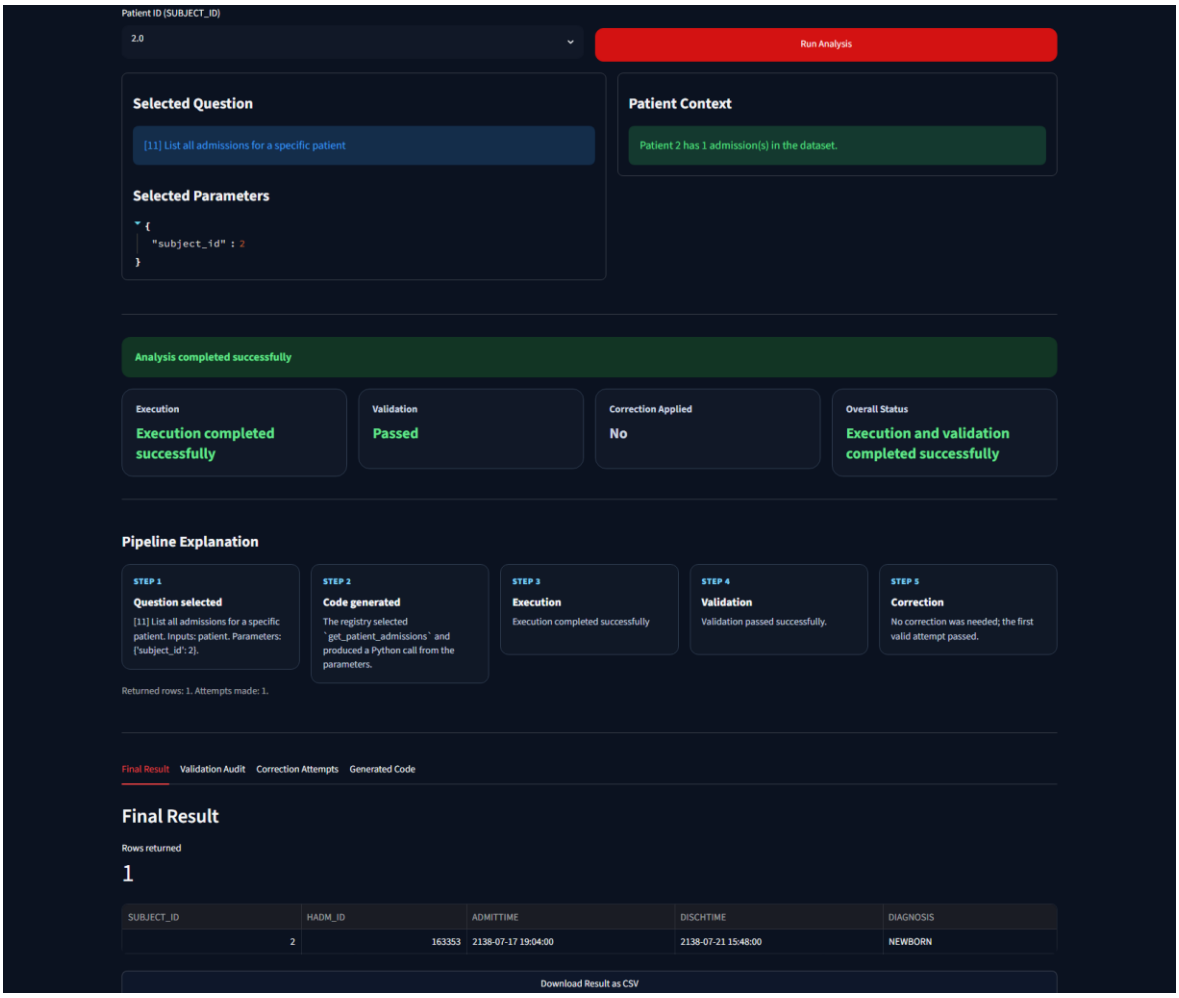


Figure 3. Question 11 (all admissions for a patient) — one of the end-to-end-tested and browser-verified questions.

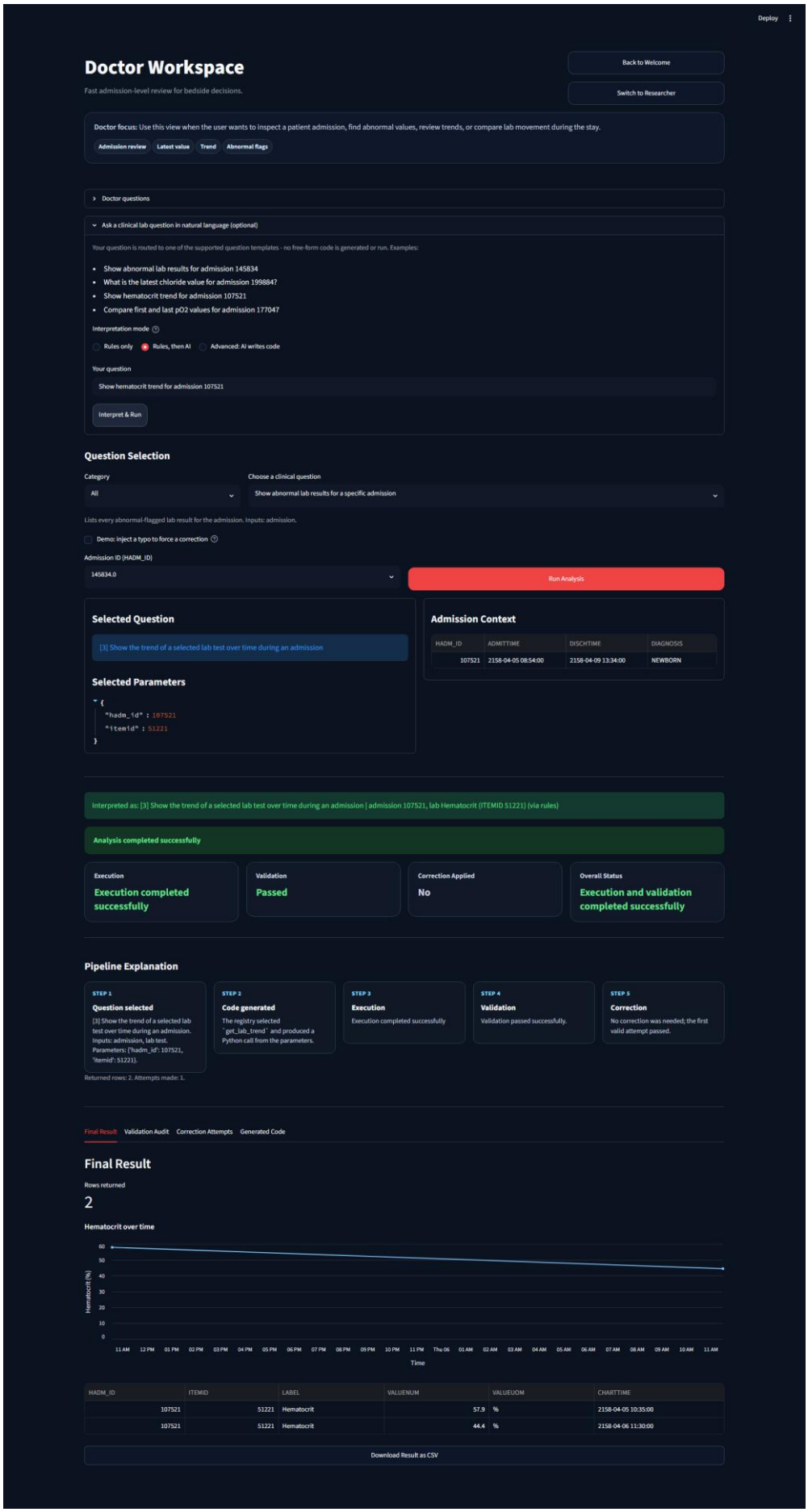


Figure 4. The controlled natural-language box: free text is routed to a supported template and its parameters, never to generated code.

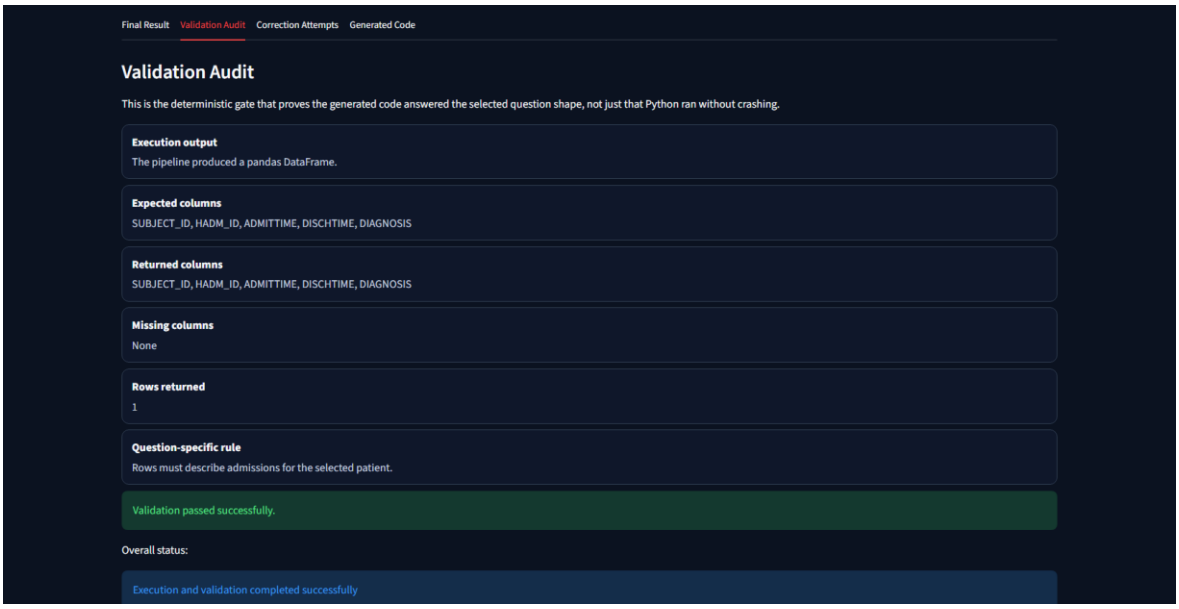


Figure 5. The Validation Audit tab, showing the structural and clinical rules that were checked and their outcome.

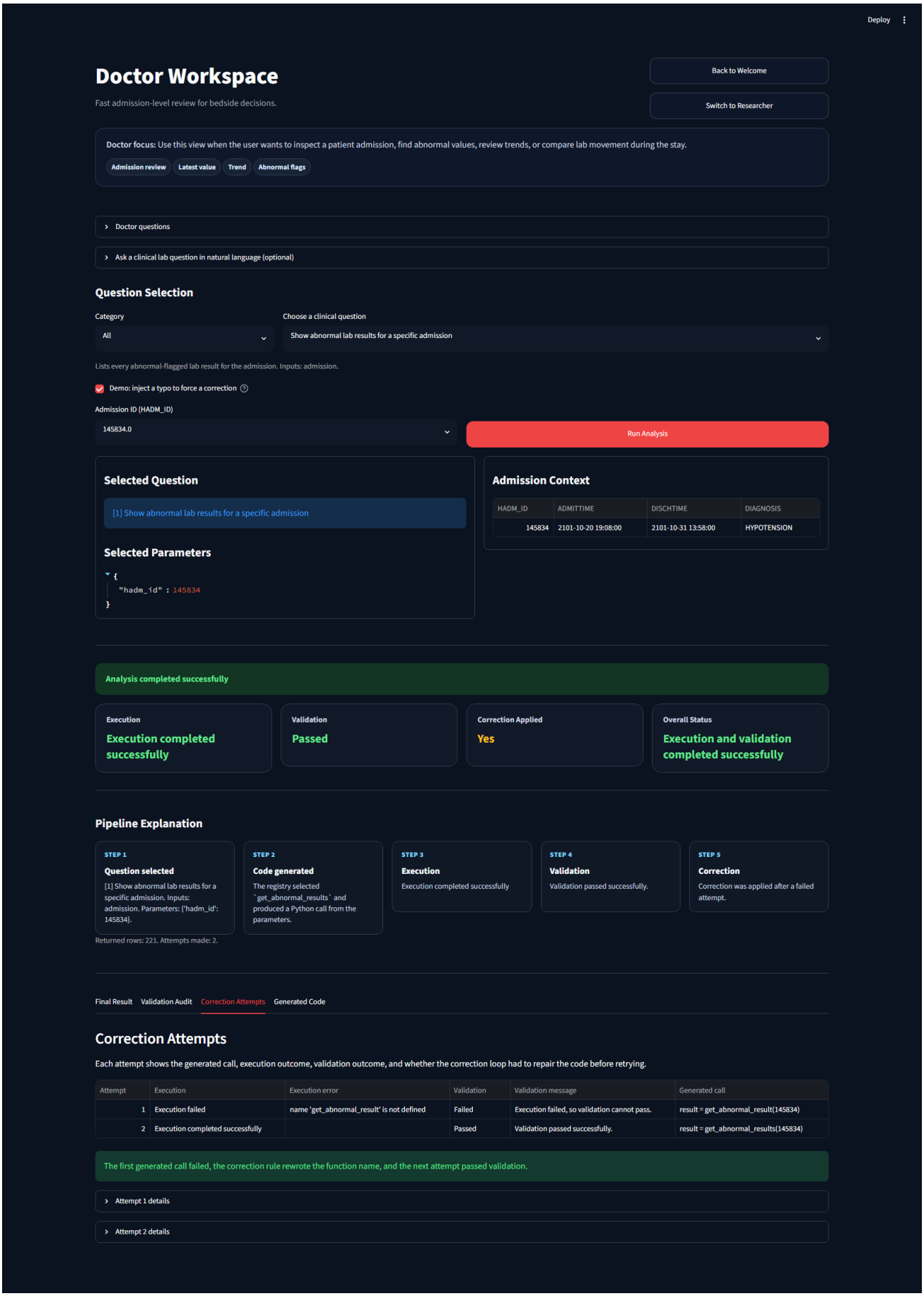


Figure 6. The Correction Attempts tab after the "inject a typo" demo: attempt 1 fails on a misspelled function name and is auto-repaired for attempt 2.

Edge cases exercised

- No-records path: a question with no matching data shows a neutral “no matching records” state rather than an error.
- Correction loop: a deliberately misspelled function name is auto-corrected and re-run (used as a live demo, since the real generator never emits a typo on its own).
- Regression guard: the typo corrector no longer corrupts the valid name get_abnormal_results_for_lab, which contains a shorter valid identifier as a substring.
- Validation failures: execution errors, a None result, a non-DataFrame result, missing columns, an empty frame, and a failing per-question rule are all handled and reported distinctly.
- Dataset-wide aggregate (Q12): per-admission counts are cross-checked against the single-admission view (Q7), and Abnormal + Normal = Total Tests is validated for every row.
- Code-gen sandbox: imports, open, dunder/underscore attributes, file/network/eval methods, loops, and syntax errors are all rejected by the AST allow-list; runtime errors, timeouts, and row caps are exercised with a fake LLM that is made to fail, be blocked, and refine.

Key findings. All 12 supported question types passed both manual verification and automated validation; 170 of 170 tests pass with 94% line coverage; every injected failure was converted into an explicit validation outcome or an automatic repair rather than a silent error; and the baseline ablations in section 3.5 show that each guardrail catches failures that would otherwise pass unnoticed. Overall, the deterministic registry path behaved consistently across every evaluated scenario, while the experimental AI mode remained safely bounded by its sandbox and validator.

Why 170 automated tests were needed

The test suite was not built to reach a large number for its own sake. The 170 tests follow a risk-based approach: every important responsibility of the system has at least one direct test, and the highest-risk paths have multiple positive and negative tests:

- Core pipeline: question selection, code generation, execution, validation, correction, and presentation.
- All 12 registry questions: required inputs, expected columns, and expected result shape.
- Failure paths: empty results, missing columns, non-DataFrame results, execution errors, invalid IDs, and validation failures.
- AI/NLP risk areas: natural-language routing, the optional LLM fallback, and sandbox blocking of unsafe generated code.
- UI behavior: end-to-end tests drive both role workspaces and all supported questions.

The suite is considered sufficient because it matches the system’s acceptance criteria rather than only measuring code coverage: the requirements say every supported question must execute, validate, and render correctly, and the tests exercise exactly that path for all 12 questions, together with the safety mechanisms that could otherwise fail silently - validation rules, correction attempts, no-record states, routing decisions, and sandbox rejection. The 94% line coverage is useful evidence, but the more important coverage is behavioral: unit tests isolate each module, pipeline tests verify the combined question-to-result flow, router tests verify natural-language mapping, sandbox tests verify that unsafe code is blocked, and end-to-end tests verify the application as a user sees it.

Why the tests are trustworthy. The tests are automated and repeatable, so the same checks run after every change; deterministic and mostly offline, so results do not depend on a live LLM response; and they include

negative tests, not only happy paths, which demonstrates that the system rejects bad or unsafe output. They are further backed by the manual evaluation examples above and by browser screenshots of the real UI.

Three kinds of coverage. Taken together, the suite provides code coverage (94% of lines, with an enforced 85% gate), bug and failure coverage (negative tests for every error path, plus regression tests that pin previously fixed bugs, such as the typo-corrector regression), and flow coverage (end-to-end tests that walk both role workspaces through all 12 questions from input widgets to rendered results).

3.5 Comparison to baselines and discussion

The project does not have an external, pre-existing system to benchmark against, so three internal baselines were used to justify each guardrail rather than assuming it a priori.

- **Unguarded execution as a baseline.** Running the same generated snippet with a plain exec() and no validation or correction step silently returns whatever the code produced—including an empty DataFrame or an output with an incorrect structure—with no indication to the user that the result may be invalid. Adding the validation layer converts such silent failures into explicit validation outcomes (for example, missing columns, empty results, or failed clinical rules), while the correction layer automatically recovers a meaningful class of failures (such as function-name typos) instead of exposing them directly to the user.
- **Rule-based routing as a baseline for natural-language input.** The default router (question_router.route_question) uses deterministic keyword and pattern matching. Comparing it with the optional LLM-assisted fallback (llm_router.py) demonstrates the expected trade-off: the rule-based router is fully reproducible and requires no external API calls, but it recognizes only phrasings covered by its predefined rules. The optional LLM fallback (used only when the rule-based routing fails and an API key is available) understands a wider range of natural-language formulations at the cost of external dependency and reduced determinism. For this reason, it is implemented as a fallback rather than the default routing mechanism.
- **The trusted registry path as a baseline for the experimental AI-writes-code path.** Across the 12 supported registry questions, the deterministic pipeline demonstrated consistent and reliable behaviour throughout the evaluation. In addition, the automated test suite (170 passing tests) verified the correctness of both the individual modules and the complete end-to-end pipeline. The optional Advanced mode trades some of this predictability for flexibility: it can answer questions outside the fixed registry, but its AST allow-list intentionally rejects unsupported language constructs (such as loops and imports), and its output is validated using the more general validate_freeform_result() function, which verifies the result's type and structure rather than applying question-specific clinical rules. This comparison supports the decision to keep the registry-driven pipeline as the trusted default and the AI-generated code path as an explicitly labelled, opt-in experimental extension. Overall, the evaluation supports the project's central claim: a small, fully specified set of clinical laboratory questions can be answered reliably through a generate → execute → validate → correct pipeline, while more open-ended LLM-generated analyses can be supported only when execution is sandboxed and every result is still verified by a deterministic validation stage before being presented to the user.

4. Development Process, Challenges and Lessons

4.1 Development phases

1. Steps 1–4 — Scope and data: froze the MVP scope (a small predefined question set only, ADMISSIONS + LABEVENTS + optional D_LABITEMS, lab rows restricted to a valid HADM_ID), chose the first 5 question types, inspected the raw MIMIC-III schema in Google Colab (missing-value counts per column, see section 3.2), and built the cleaned, filtered, merged working dataset.
2. Steps 5–16 — First MVP: defined the 5-module architecture (Question Selection, Data Preparation, Code Generation, Validation and Correction, Result Presentation), picked the Python + pandas + Streamlit stack, and implemented each module — first against mock/sample DataFrames to validate the interfaces, then wired to the real dataset — arriving at a first working end-to-end console pipeline before building the Streamlit UI.
3. Step 17 — Registry refactor and polish: replaced the 5 hard-coded questions with a single config-driven QUESTION_REGISTRY (questions.py) that every other module reads from, added 6 further question types (11 total), and polished the UI — a category filter, per-question dynamic parameter widgets, a trend chart, a status banner that reflects success / no-records / failure, and a pinned dark theme.
4. Step 18 and follow-ups — Final deliverables and extensions: wrote the project report, slide deck, and demo script; added a 12th, param-less, dataset-wide aggregate question; added controlled (then optionally LLM-assisted) natural-language routing; added the experimental, sandboxed “Advanced: AI writes code” agent loop inspired by concepts presented in AgentCoder and Self-Edit; and added the role-based Doctor / Researcher workspace UI.

4.2 Challenges and how they were resolved

The cleaned dataset was too large for git

cleaned_merged_dataset.csv (~130 MB) exceeds GitHub's 100 MB per-file limit, and Git LFS was judged not worth its quota/setup overhead for a course project. Resolution: the dataset is kept out of version control (.gitignore) and the app downloads it automatically on first run from a shared location, with a documented manual fallback.

A naive typo-correction rule corrupted a valid function name

The first version of the rule-based corrector used plain substring replacement, so the fix for the typo get_abnormal_result silently rewrote the valid, longer identifier get_abnormal_results_for_lab (which contains that substring) into something else. Resolution: the corrector now matches on whole identifiers only, using regex word boundaries, and a regression test pins this behaviour.

Theme inconsistency made the UI unreadable for some users

Headers and cards were unreadable when a viewer's system theme did not match the app's assumptions. Resolution: the Streamlit theme was pinned explicitly in .streamlit/config.toml so the app renders identically regardless of the viewer's OS/browser theme.

How much autonomy to give generated code

The team's initial scope decision was to exclude any LLM code generation from the MVP, since free-text/unbounded questions would break the validator's assumption of known expected columns and could execute arbitrary code. This decision was later revisited, but only by adding a second, clearly optional and off-by-default path, gated by a purpose-built sandbox (AST allow-list, restricted builtins, worker-thread

timeout, row cap) built before any LLM-authored code was allowed to run at all — rather than relaxing the guarantees of the trusted registry path.

Keeping a growing question set consistent across modules

As the question count grew from 5 toward 12, keeping the UI, code generator, validator, and pipeline in sync by hand became error-prone. Resolution: the registry refactor introduced questions.py as the single source of truth for all supported questions. The user interface, code generation, validation, and execution pipeline all derive their behaviour from this registry, so adding a new question now requires updating only one central definition.

Why the course website was not built in Google Sites

The first and most practical reason is that Google Sites is visually basic: its templates are plain, customization is shallow, and it offers no building blocks for the components the site relies on. Because the course website was treated as part of the graded software deliverable, the team chose a standalone HTML/CSS site for full design control, safer file delivery, and an engineered workflow:

- Limited design and features. The rubric grades readability, navigation, uniformity, and aesthetics, and a review of 39 past course sites showed that Google Sites submissions usually share the same generic template. Standalone HTML/CSS allowed a purpose-built design — the clinical theme, stat tiles, coverage bars, an SVG pipeline diagram, styled tables, and a captioned screenshot gallery — none of which Google Sites components can express.
- Self-contained files remove permission risk. Google Sites teams often attach materials as Google Drive links, and several past sites had permission walls or dead links. Here, every PDF and PPTX sits inside the submitted folder with relative links, so the site works offline from the unzipped folder.
- Delivery is more robust. The zip is graded exactly as built; a hosted site depends on account ownership and permissions after submission (one past course site is now login-walled and effectively lost). A standalone zip cannot rot, lose access, or accidentally become private.
- Precedent supports the choice. Strong previous standalone submissions exist in the course, and course staff republish standalone zips on GitHub Pages, so this is an accepted path.
- The workflow matches a software-engineering seminar. The site is version-controlled in Git; check_site.py automatically verifies internal links, relative links, image alt text, and hygiene rules, and package.py reproducibly builds the required submission zip.

The honest trade-off: Google Sites offers free hosting and easy WYSIWYG collaboration. Collaboration was covered through GitHub, and a standalone submission needs no hosting at all.

4.3 Lessons learned

- A single source of truth (the question registry) is worth the refactor: it removes an entire class of drift bugs between the UI, the generator, and the validator, and it made every later addition (questions 6–12, the role-based UI, the code-gen agent) a localized change.
- Deterministic validation plus a bounded correction loop is a cheap, high-value safety net — it turns silent or malformed output into an explicit, auditable status without needing a model call for the common failure modes.
- Sandboxing has to come before capability: the AST allow-list, restricted builtins, timeout, and row cap were designed and tested before any LLM-authored code was allowed to execute, not retrofitted afterward.

- Lightweight, in-repository orchestration was preferable to adopting a heavier multi-agent framework (such as CrewAI, AutoGen, or LangGraph) for the experimental code-generation loop. This decision kept the implementation lightweight, easier to debug, and more appropriate for the project's educational scope. The overall design is also consistent with the ideas presented in AgentCoder, where clearly separated responsibilities are emphasized without requiring complex orchestration frameworks.

5. Summary and Future Work

5.1 Conclusions

The project demonstrates that a small, fully specified set of clinical laboratory questions can be answered reliably and transparently by a generate → execute → validate → correct → present pipeline built on a single-source-of-truth question registry, and that the same pattern can be extended, cautiously, toward more open-ended LLM-authored queries as long as execution is sandboxed and every result still passes through a deterministic gate before it is trusted. Across 12 supported questions and 170 automated tests (94% line coverage), the deterministic path demonstrated consistent and reliable behaviour across all evaluated registry questions, while the optional Advanced mode demonstrated that the same pattern can be extended to more open-ended queries—at the cost of a weaker, generic validator and a narrower set of expressible code.

5.2 Advantages

- Reliable and reproducible by default: the trusted path never calls an external model and is fully deterministic and test-covered.
- Transparent and auditable: every run shows the generated code, the validation outcome, and the full correction history, not just the final answer.
- Maintainable: the config-driven registry keeps the UI, generator, validator, and pipeline in sync; adding a question is a one-place change.
- Safe-by-default extensibility: free-form, LLM-authored code is possible but strictly opt-in and sandboxed, never the default execution path.
- Usable for two distinct audiences: the role-based Doctor / Researcher workspace surfaces the subset of questions and framing relevant to each user without hiding the shared, trusted pipeline underneath.

5.3 Disadvantages and limitations

- Restricted, by design, to a fixed set of 12 predefined question types; it cannot answer an arbitrary clinical question without either extending the registry or switching to the weaker Advanced mode.
- Operates on a single static, local CSV rather than a live SQL backend, so it does not reflect data that changes after the file is prepared.
- The rule-based corrector only repairs a small, explicit set of known function-name typos; it cannot fix arbitrary bugs, unlike a learned, fault-aware editor such as Self-Edit (section 7).
- The AST allow-list in the experimental mode blocks loops, imports, and several pandas methods, so it will reject some valid, more complex analyses along with unsafe ones.
- No authentication or multi-user state; all valid-but-empty results are reported uniformly as “no records” without deeper diagnostics.

5.4 Limitations of the evaluation

- Evaluation is on one dataset (a MIMIC-III derived merge) and one clinical domain (lab events), so generalization to other tables or hospitals is untested.
- There is no external baseline system or a formal comparison against clinicians' own ad-hoc queries; the “baselines” used in section 3.5 are internal ablations of the pipeline's own guardrails.
- The experimental LLM code-generation mode was exercised in automated tests mostly with a fake/mock LLM to test control flow, rather than benchmarked at scale against real models the way AgentCoder or Self-Edit benchmark theirs.

5.5 Future work

- Broaden the question registry, or pair a more capable LLM-assisted router with the same validation guarantees, so more clinical questions can be answered through the trusted path rather than the weaker Advanced mode.
- Replace the static CSV with a live SQL backend so results reflect current data and larger cohorts can be queried efficiently.
- Strengthen the guarded LLM code-generation path: a broader, still-safe AST allow-list; benchmarking across multiple LLM providers; and a learned, fault-aware correction step in the spirit of Self-Edit instead of only rule-based typo fixing.
- Run a small user study with clinicians and researchers comparing the role-based workspace against ad-hoc querying, to validate the usability claims with real users rather than internal review.
- Add authentication and multi-user session state if the system moves beyond a single-user course demo.

6. User Manual

6.1 System requirements and dependencies

- Python 3.10 or later (developed and tested on Python 3.13).
- The uv package manager (https://docs.astral.sh/uv/), used to install and pin all dependencies from pyproject.toml / uv.lock.
- Core dependencies (installed automatically by uv sync): pandas >= 2.2, streamlit >= 1.36, httpx >= 0.27, python-dotenv >= 1.0.
- Development-only dependencies: pytest, pytest-cov, playwright.
- Optional: an API key for Anthropic, OpenAI, or Google Gemini, only if the optional LLM-assisted routing or Advanced AI-writes-code mode is desired. The system is fully functional with no key at all.
- Optional .env file: a local configuration file that links the app to the selected LLM provider by storing one key such as ANTHROPIC_API_KEY, OPENAI_API_KEY, or GOOGLE_API_KEY. It enables the optional AI features, is not needed for the 12 predefined questions, and should not be committed to version control.
- The cleaned dataset cleaned_merged_dataset.csv (~130 MB) — downloaded automatically on first run, or placed manually in the project root next to app.py. (https://drive.google.com/file/d/1AwaAmPRDq_kqRmhaVAPwPAR4YpaEsElb/view)

6.2 Installation and running the system

1. Get the code: clone the repository with git clone https://github.com/alonengel/seminar-project.git and enter the project folder — or download the repository as a ZIP from GitHub and extract it (no git required).
2. Install uv, then from the project root run: uv sync — this creates a local virtual environment and installs every pinned dependency.
3. (Optional) copy .env.example to .env and fill in one provider API key to enable LLM-assisted routing and the Advanced code-generation mode; leave it empty to run in fully rule-based / deterministic mode.
4. Ensure cleaned_merged_dataset.csv is present in the project root, or simply start the app and let it download the file automatically on first launch.
5. Launch the interactive UI: uv run streamlit run app.py, then open the printed local URL (typically http://localhost:8501).
6. Alternatively, run a single question end-to-end from the command line: uv run python main_pipeline.py.
7. Run the automated test suite (170 tests, coverage report): uv run pytest.

6.3 Required inputs, by question

Question	Required inputs
1, 4, 7, 9 — abnormal results / all tests / counts / distinct abnormal tests for an admission	Admission ID (HADM_ID)
2, 3, 5, 6, 8 — latest value / trend / first-vs-last / summary stats / abnormal results for a lab	Admission ID + lab test (name or ITEMID)
10 — values within a date range	Admission ID + lab test + start/end date
11 — all admissions for a patient	Patient ID (SUBJECT_ID)
12 — abnormal vs normal counts across all admissions	None (runs across the whole dataset)
Natural-language box (any question above)	A free-text sentence, e.g. “Show hematocrit trend for admission 107521”

6.4 Operation steps

Automatic, on startup

- The dataset is loaded once (data_prep.py) and, if missing, downloaded from the configured source.
- The registry-driven function context used by the code generator/executor is built from questions.py.

Manual, per query

1. From the Welcome screen, choose the Doctor or Researcher workspace (or use the direct question dropdown).
2. Pick a category and a question, or expand “Ask a clinical lab question in natural language” and type one.

3. Fill in the parameter widgets that appear for the chosen question (only the parameters that question needs are shown).
4. Optional: choose an interpretation mode for natural-language input (Rules only / Rules, then AI / Advanced: AI writes code), or tick the “inject a typo” demo checkbox (available for questions 1–5) to see the correction loop fire.
5. Click Run Analysis (or Interpret & Run for the natural-language box).

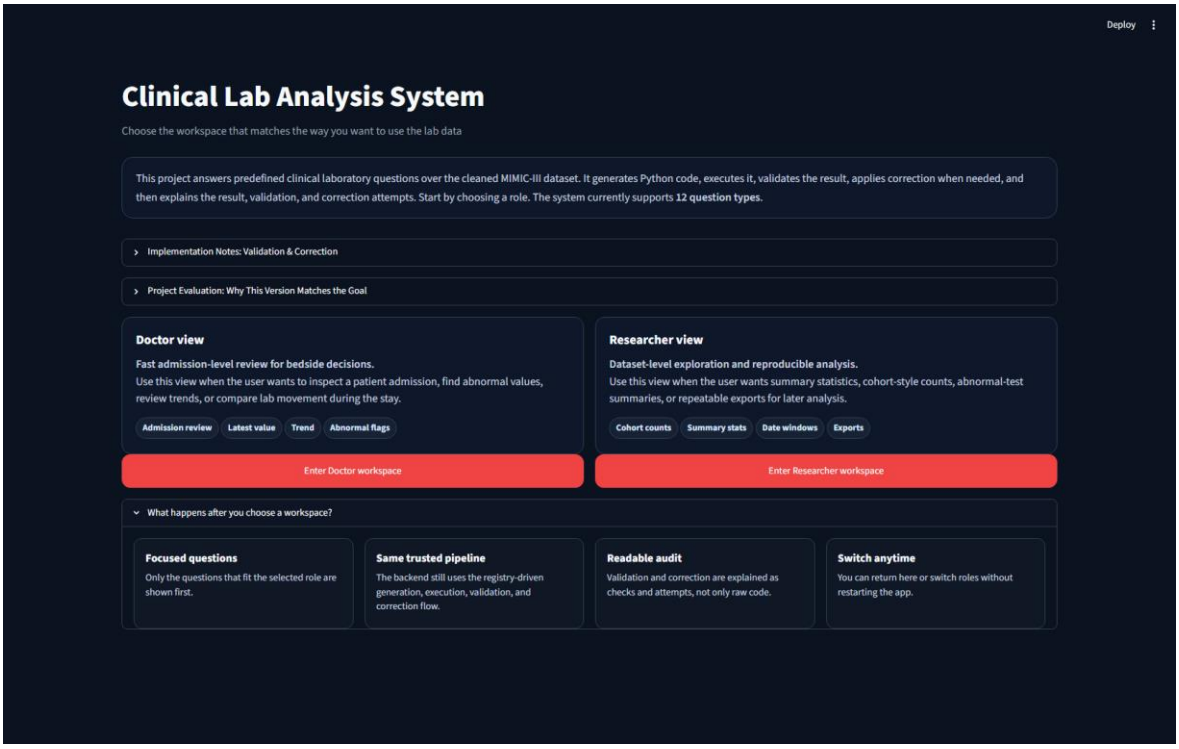


Figure 7. Step 1 of the manual flow — the Welcome screen, where the user chooses the Doctor or Researcher workspace.

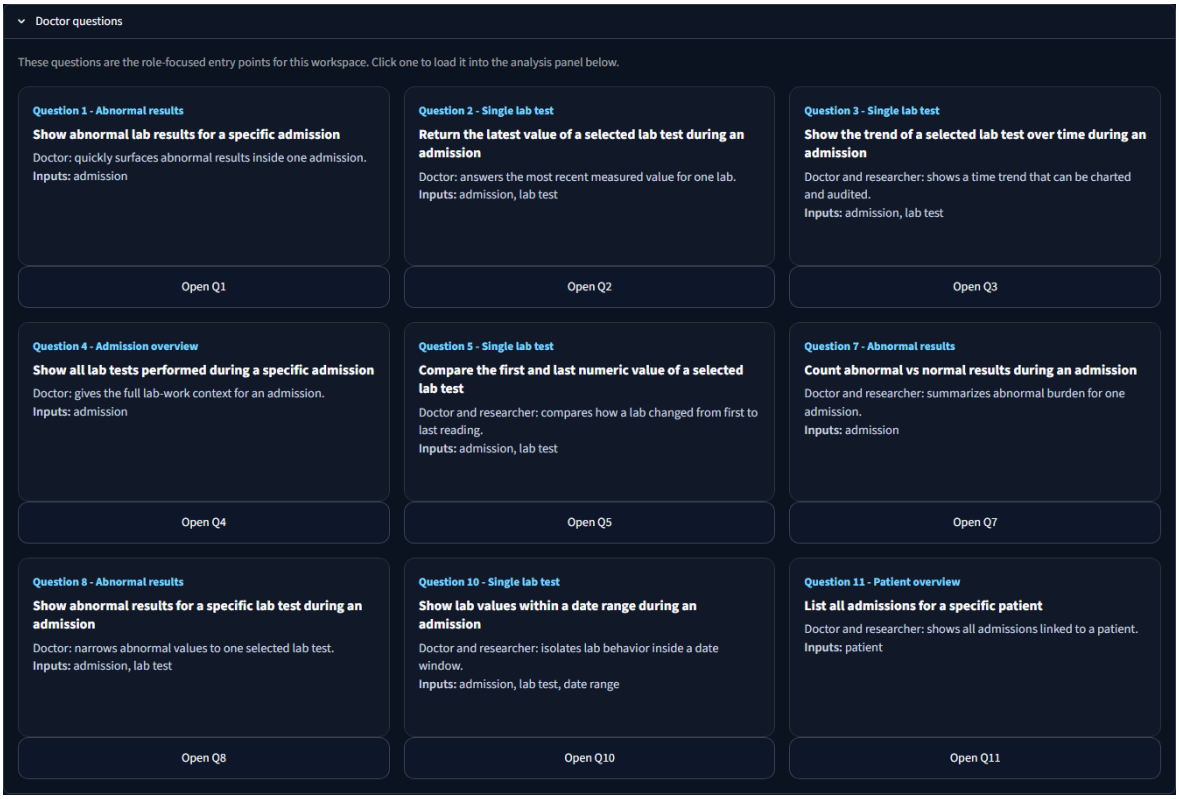


Figure 8. Step 2 — each workspace surfaces its role's questions as cards (category, description, and required inputs); one click loads a question into the analysis panel.

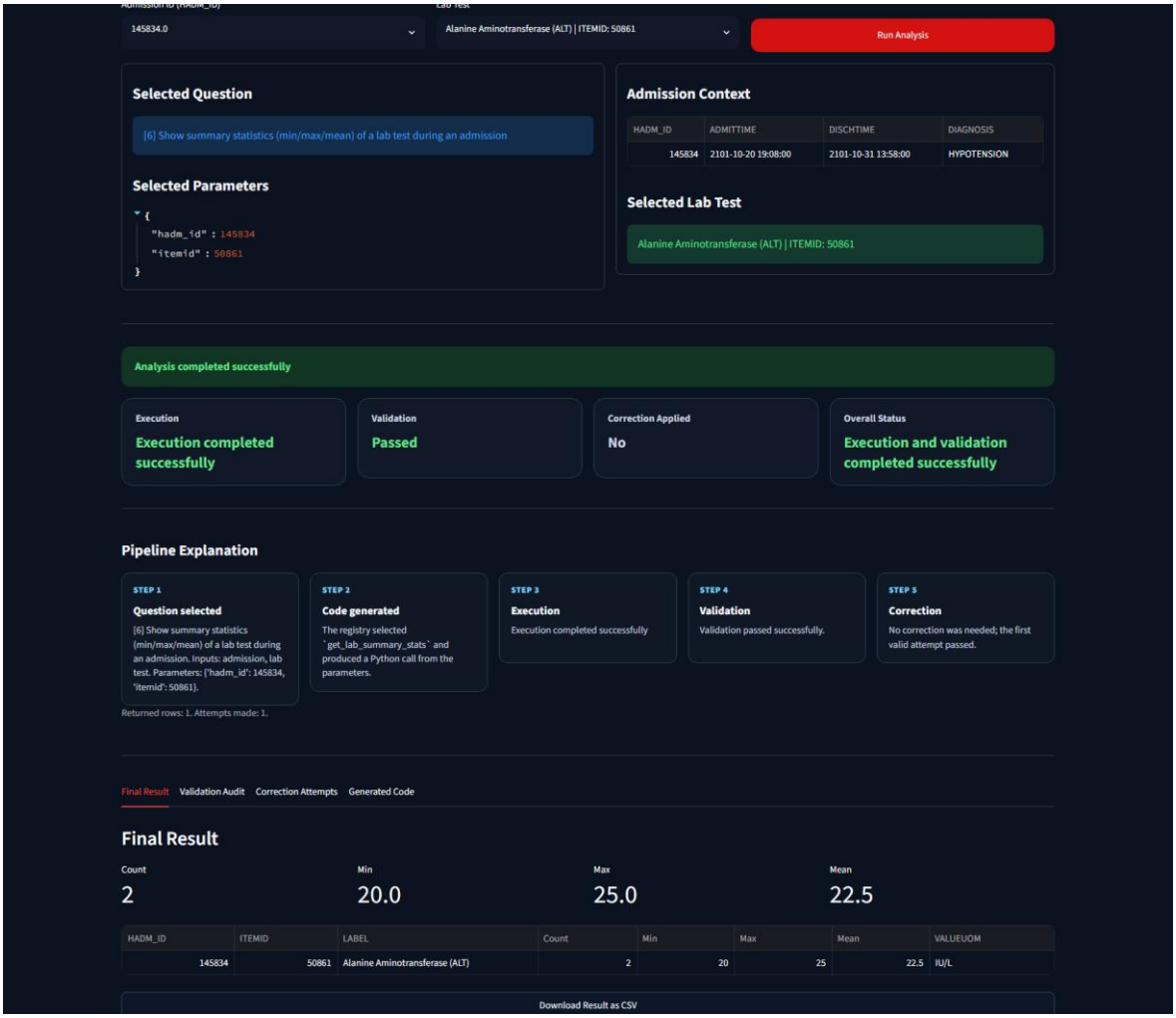


Figure 9. Steps 3–5 of the manual flow — a complete run: the selected question's parameter widgets (admission and lab test), the Run Analysis button, and the resulting status banner, summary metric cards, and output tabs.

6.5 Outputs

- A colour-coded status line (green = passed, blue = no matching records, red = failed).
- Summary metric cards appropriate to the question (e.g. latest value, first-vs-last difference, count/min/max/mean, abnormal counts).
- A trend line chart for time-series questions (trend and date-range questions).
- Final Result tab: the result table with a "Download Result as CSV" button.
- Validation Audit tab: which structural/clinical rule was checked and whether it passed.
- Correction Attempts tab: every attempt's code, execution outcome, and validation outcome ("No correction attempts were needed" in the normal case).
- Generated Code tab: the exact Python snippet that was executed to produce the result (for the Advanced mode: Final Result / AI Process / Generated Code / Attempts tabs).

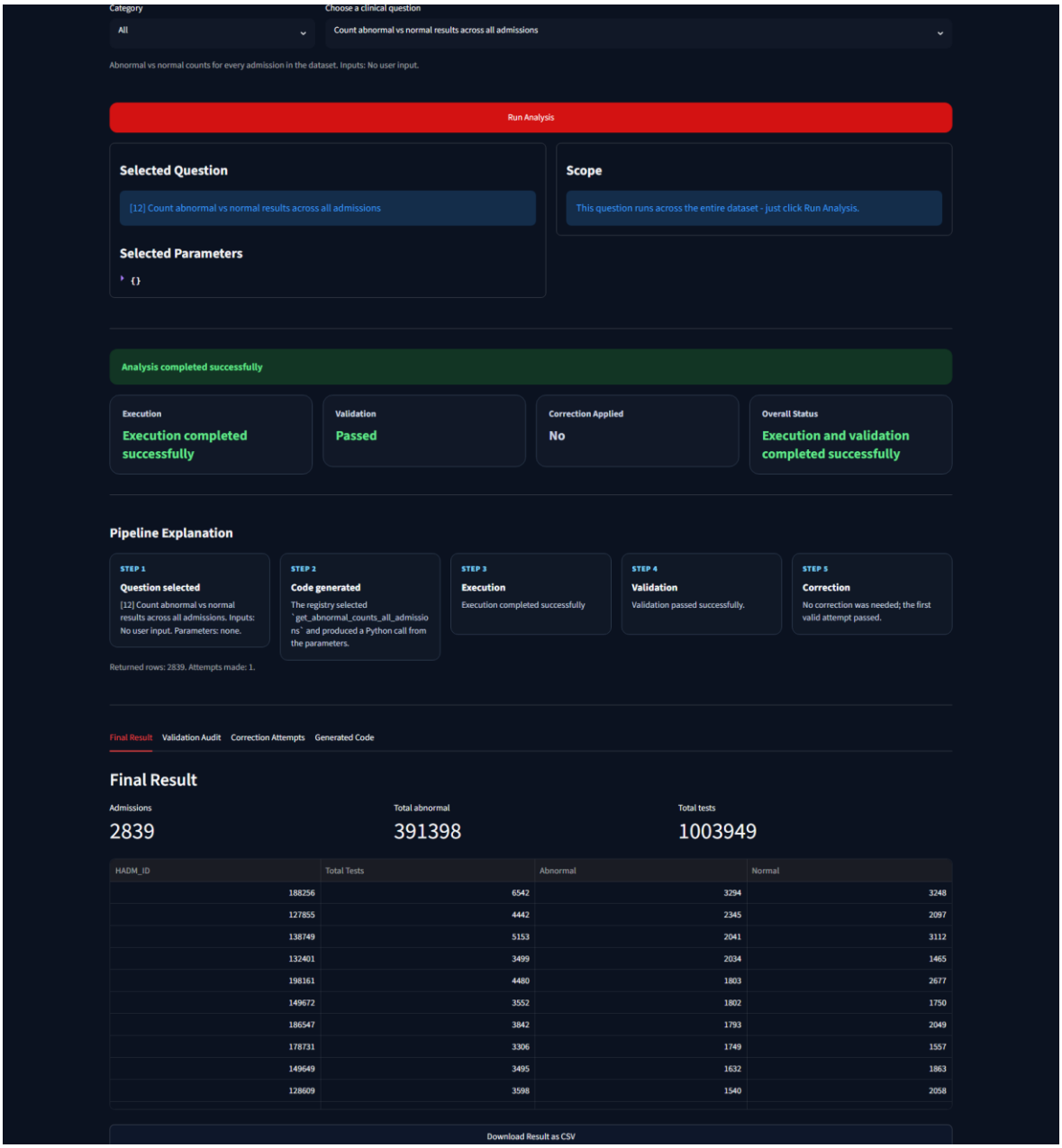


Figure 10. The outputs of a dataset-wide run (Q12) in the Researcher workspace — the colour-coded status line, summary metrics, and the Final Result / Validation Audit / Correction Attempts / Generated Code tabs described above.

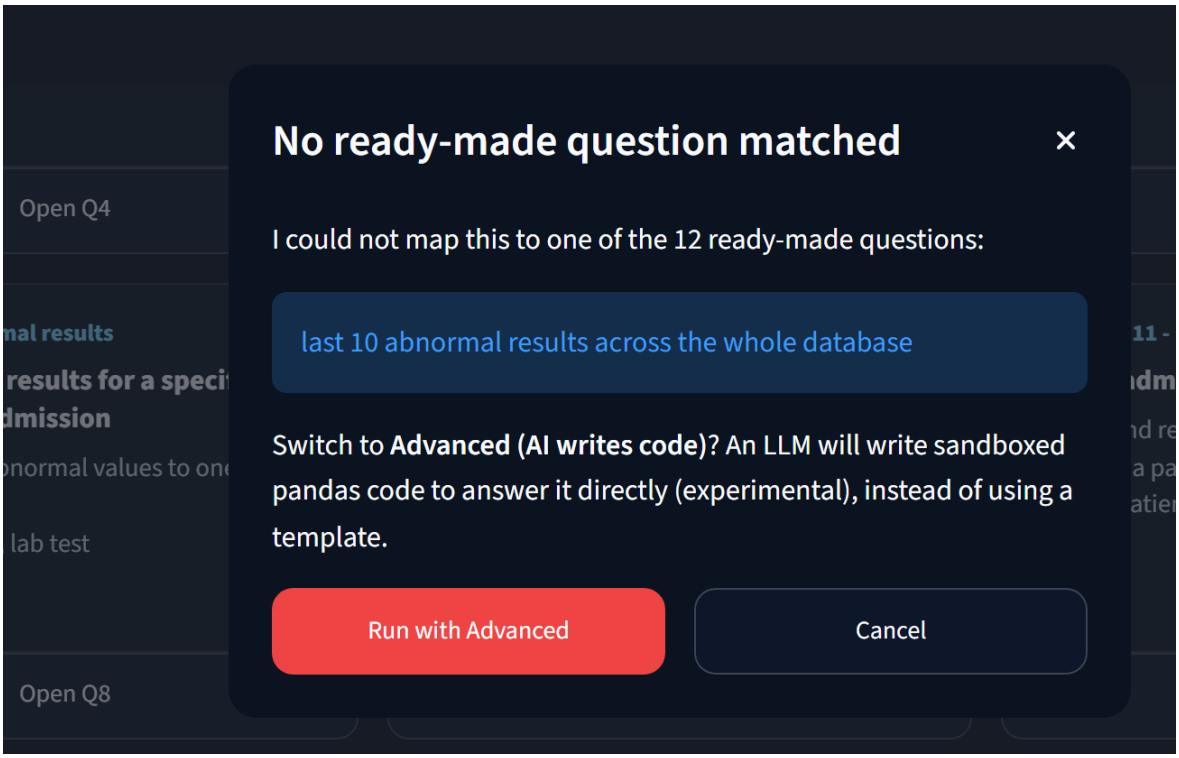


Figure 11. The escalation dialog: when neither the rules nor the AI-assisted router can match a typed question (here: “last 10 abnormal results across the whole database”), the app offers to re-run it in the Advanced mode — or cancel — instead of dead-ending.

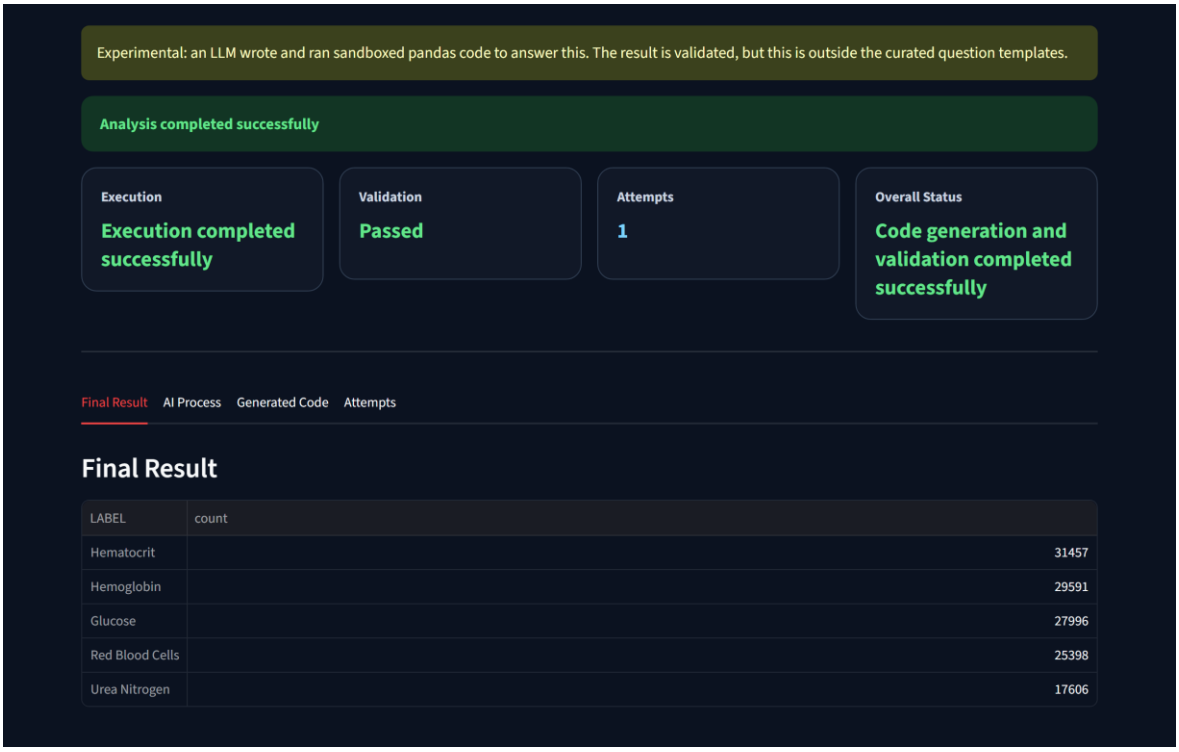


Figure 12. The Advanced (“AI writes code”) mode answering a free-form question (“the 5 most common abnormal lab tests across all admissions”): the LLM-written pandas snippet ran inside the sandbox and passed validation on the first attempt; the result appears with the Final Result / AI Process / Generated Code / Attempts tabs.

6.6 Repository / code structure

Path	Contents
app.py	Streamlit UI
main_pipeline.py	End-to-end console pipeline
questions.py	Question registry (single source of truth)
question_router.py, llm_router.py, llm_client.py	Controlled / optional LLM-assisted natural-language routing
data_prep.py	Dataset load + per-question data functions
code_generation.py	Template code generation
execution.py	Registry-path execution wrapper
validation.py	Output validation
correction.py	Rule-based correction loop
result_presentation.py	Console presentation
sandbox.py, code_agent.py, agent_pipeline.py, presentation_agent.py	Experimental, sandboxed LLM code-generation mode
tests/	170 unit + end-to-end tests
docs/	PRD, PLAN, TODO, REPORT, DEMO, SLIDES, TEST_REPORT, decision log, screenshots
pyproject.toml, uv.lock	Dependency management (uv)

7. Articles Read

7.1 Abstracts

AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation

Huang, D., Zhang, J. M., Luck, M., Bu, Q., Qing, Y., & Cui, H. (2023, revised 2024). arXiv:2312.13010.

https://arxiv.org/pdf/2312.13010

The paper addresses a persistent weakness of LLM-based code generation: a single model asked to both write and check its own code tends to balance these two jobs poorly, generating code that looks plausible but fails on real test cases. The authors propose AgentCoder, a multi-agent framework with three specialized, cooperating roles: a programmer agent that writes and refines code, a test designer agent that independently generates test cases for that code, and a test executor agent that runs the code against those tests and reports concrete feedback (pass/fail, error messages) back to the programmer. This separation of concerns — write, independently test, execute and report — lets the programmer iterate using real execution feedback instead of the model's own untrustworthy self-assessment. The main contribution is showing that this lightweight three-agent loop beats both single-agent prompting and heavier multi-agent frameworks: across 9 code-generation models and 12 enhancement approaches, AgentCoder with GPT-4 reaches 96.3% and 91.8% pass@1 on HumanEval and MBPP respectively, versus 90.2%/78.9% for the prior state of the art, while using roughly a third of the token overhead — evidence that more agents/rounds are

not automatically better, and that a small, well-separated set of roles with real execution feedback is what actually drives the improvement.

Self-Edit: Fault-Aware Code Editor for Code Generation

Zhang, K., Li, Z., Li, J., Li, G., & Jin, Z. (2023). Proceedings of ACL 2023.

https://aclanthology.org/2023.acl-long.45.pdf

This paper starts from the same observation that motivates AgentCoder — LLMs generate code with real but limited accuracy on competitive-programming-style tasks — but proposes a lighter, generate-and-edit pipeline instead of a multi-agent one. Self-Edit has three steps: an LLM generates an initial program from the problem description; the program is executed on the example test case(s) given with the problem, and any error or wrong output is wrapped into a structured "supplementary comment" (e.g. the exact exception and a hint at the fix); and a separate, purpose-trained fault-aware code editor model then rewrites the code conditioned on the original program, the problem description, and that comment. Evaluated across 9 LLMs (110M–175B parameters) on the APPS and HumanEval benchmarks, Self-Edit improves average pass@1 by 89% on APPS-dev, 31% on APPS-test, and 48% on HumanEval relative to generating directly from the LLM, and does so with a constant, small sampling budget — outperforming reranking-based post-processing methods that instead sample many candidates and pick one. The key contribution is demonstrating that execution feedback, distilled into a compact structured comment, is enough signal for a dedicated editor model to reliably repair a large fraction of an LLM's own coding mistakes.

7.2 Contribution of each article to the project

AgentCoder → the optional Advanced code-generation mode

AgentCoder's generate → test → execute → feedback loop, and specifically its finding that a small set of clearly separated roles beats a heavier multi-agent framework, influenced the design of the project's experimental "Advanced: AI writes code" path. agent_pipeline.py implements the same generate → execute → validate → refine shape: code_agent.py plays the programmer-agent role, sandbox.py plays the (deterministic, non-LLM) executor role, and validation.validate_freeform_result plays a lightweight stand-in for the independent test/validator role. Following AgentCoder's own critique of frameworks such as MetaGPT or ChatDev — that stacking more agents mainly adds token and coordination overhead once the necessary roles exist — the project deliberately kept this orchestration lightweight and in-repo rather than adopting CrewAI, AutoGen, or LangGraph, reusing two roles (execution, validation) that the project's own deterministic modules already provided.

Self-Edit → the rule-based correction loop

Self-Edit's core idea — run the generated code, turn the execution result into structured feedback, and use that feedback to edit the code before resubmitting —served as the conceptual model for correction.py, whose own module docstring describes it as "Correction module (a simplified, rule-based interpretation of the Self-Edit approach)". The project's corrector is a deliberately simplified, rule-based analogue of Self-Edit's learned fault-aware editor: instead of a trained neural editor conditioned on an execution comment, correct_generated_code() applies a small, explicit table of known function-name-typo fixes to the code and re-runs it, still following Self-Edit's generate → execute → (structured feedback) → edit → resubmit shape, but with a deterministic rule set in place of a learned model — appropriate for a bounded, fully-specified question registry where the space of likely code-generation faults is small and known in advance. The paper's broader finding, that execution feedback alone (without re-sampling from the LLM) is enough signal

to repair a meaningful share of generation errors, is the justification for keeping correction rule-based and cheap rather than requiring another model call every time execution fails.

Appendix A: LLM Usage, Prompts, and Safety

This appendix documents how large language models (LLMs) were used in the project — both inside the running system and during development — including the exact prompts. Anything done only partially is stated explicitly.

A.1 The in-system prompts (verbatim)

Two prompts run inside the system; both are reproduced verbatim from the source code.

Programmer-agent system prompt (code_agent.py). "You are a careful clinical-data analyst. Write a SINGLE Python snippet that uses pandas to answer a question about a lab-results DataFrame. Strict rules: Use ONLY the provided DataFrame df and the pd module. Assign the final answer to a variable named result (a DataFrame, Series, or scalar). Do NOT import anything, read or write files, or use input/eval/exec/query. Do NOT access private/underscore attributes and do NOT use loops; prefer vectorized pandas. You may add one short '# plan: ...' comment, then the code. Output code only, no prose." The user message attaches the DataFrame schema (column names and dtypes), up to three sample rows, the question, and — on a retry — the previous attempt's error or validation feedback ("Your previous attempt failed. Fix it.").

Router system prompt (llm_router.py). "You translate a clinical lab question into ONE supported question template. Reply with ONLY a JSON object (no prose, no markdown fences) … Pick question_id from the catalog ONLY IF that template precisely answers the question. If no template truly fits — for example the user asks for specific rows, the latest/last N results, a top-N or ranking, or anything the catalog does not cover — set question_id to 0 instead of forcing an approximate match. … Use null for anything not stated. Never invent IDs." The user message attaches the 12-question catalog (id, label, and inputs per question) and the user's sentence.

A.2 Safety by prompt design — and why prompts are never the only guard

The programmer-agent prompt constrains the model to the provided df and pd only, requires the answer in a result variable, and forbids imports, file or network access, eval/exec/query, private attributes, and loops; it also asks for a one-line plan comment, which the UI surfaces in the AI Process tab. The router prompt constrains the model to JSON-only output over a closed catalog, gives it explicit permission to answer "no match" (question_id 0) rather than force-fit, and instructs it to never invent IDs. Crucially, the prompts are only the first layer of defense: every generated snippet is still parsed against the AST allow-list, executed in the restricted sandbox with a timeout and row cap, and gated by the deterministic validator before anything is displayed (sections 2.3 and 3.3). A prompt violation therefore fails safely — a behavior exercised directly in the automated tests with a deliberately misbehaving fake LLM.

A.3 LLM use during development

Claude (Anthropic) was the only model used during development. Multi-provider support (OpenAI, Google) exists in llm_client.py and is selected by whichever API key is configured, but no formal cross-model comparison was performed — the alternative providers were exercised only through mocked responses in the automated tests.

Development-time use followed a repeatable loop rather than ad-hoc prompting: propose (brainstorming candidate questions, code structure, or text with Claude) → inspect and edit manually → validate (against the dataset's real columns, the engineering criteria of section 3.3, or the automated test suite) → accept or iterate. Concrete examples: the 12-question candidates were brainstormed with Claude across iterative sessions and filtered against the health-portal research and the four acceptance criteria (section 3.3); the correction module was designed against the Self-Edit loop (section 2.3); and parts of the report and website text were drafted with Claude and then reviewed and edited by the team.

Stated limitations, explicitly: verbatim transcripts of the development sessions were not preserved, so the prompts above are the complete in-system prompts while development prompts are described by role rather than quoted; and no quantitative model comparison was run — provider flexibility was implemented, not benchmarked.